

A dark blue vertical bar on the left side of the page. A teal arrow points to the right from this bar, containing the date.

10/1/2026

TEKRA Studio

Технічне завдання: життєвий
цикл, тестування, Production та
управління Robot'ом

Several thin, dark blue wavy lines that originate from the bottom left and curve upwards and to the right.

Oleksii Kulinskyi
TEKRA

Зміст

1. Мета	3
2. Загальна архітектура	4
2.1. Studio	5
2.2. Orchestrator	5
2.3. Assistant	6
3. Studio	7
3.1. Загальна модель	7
3.2. Рівні Studio	8
3.3. Custom components	8
4. Robot як основна сутність	10
4.1. Основні атрибути Robot'a	10
4.1.1. Robot ID	10
4.1.2. Metadata Robot'a	13
4.1.3. Version	14
4.1.4. Versioning rules	15
4.1.5. Структура Version[-s]	16
4.1.6. Lifecycle Robot'a	17
4.1.7. Authorization	19
4.1.8. Availability	20
4.1.9. Health	21
4.1.10. Active Production Version	22
4.1.11. Rollback	23
4.1.12. Test Period	24
4.1.13. TEST execution	25
4.1.14. TEST Period expiry	25
4.1.15. TEST → PROD	26
4.1.16. Execution	28
4.1.17. Структура Execution	28
4.1.18. Execution Status	29
4.2. Повна модель Robot → Version → Execution	30

4.3.	Принцип розділення станів.....	32
4.4.	Visibility.....	32
4.5.	Permissions.....	33
4.6.	TEST → PROD interactions.....	34
4.7.	Orchestrator.....	35
4.7.1.	Функції Orchestrator.....	36
4.7.2.	Pre-run checks.....	37
4.7.3.	Priority.....	38
4.7.4.	Admin Panel.....	38
4.7.5.	Audit.....	40
4.8.	Assistant.....	42
5.	Приклад lifecycle.....	44
6.	Приклад TEST.....	47
7.	Data model.....	48
8.	Responsibility matrix.....	51
9.	Acceptance Criteria.....	52
10.	Невирішені design decisions.....	54
11.	Фінальна концептуальна модель.....	56

1. Мета

Документ визначає цільову функціональну та логічну модель роботи Robot'a, його версій, тестування, переведення у PROD та виконання.

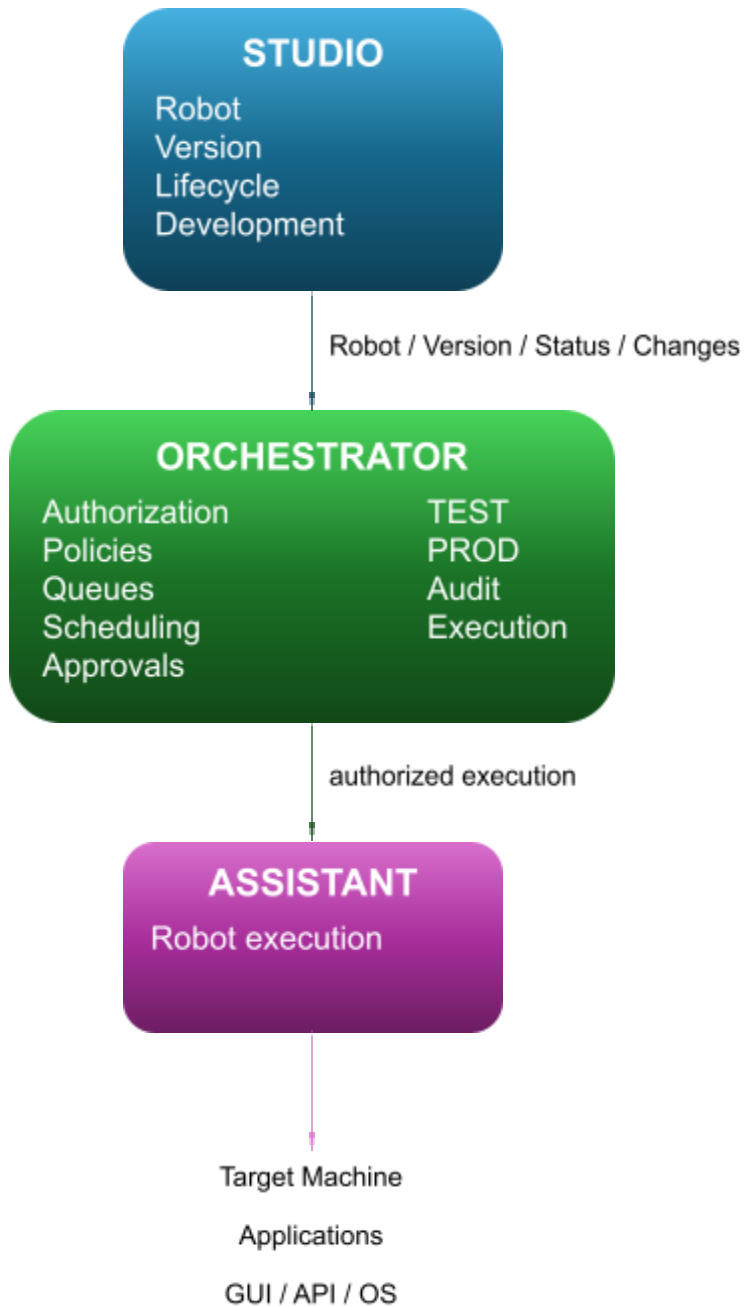
Система повинна забезпечувати:

- створення та редагування Robot'a у Studio;
- розробку та тестування Robot'a;
- багаторазове тестове виконання без необхідності погодження кожного запуску;
- створення та зберігання версій Robot'a;
- контрольований перехід конкретної Version у PROD;
- централізований контроль виконання через Orchestrator;
- виконання Robot через Assistant на цільовій машині;
- розділення lifecycle, authorization, availability та health;
- однозначне визначення конкретної Version, яка виконується;
- централізоване управління TEST та PROD;
- розмежування видимості та прав доступу;
- пріоритезацію PROD-виконань;
- повний аудит критичних дій.

Ключовий принцип: Lifecycle, Authorization, Availability, Health, Version та Execution Status - це різні характеристики Robot'a і не повинні об'єднуватися в один Status.

2. Загальна архітектура

Система складається з трьох основних компонентів:



2.1. Studio

Studio - web-застосунок, доступний через браузер. Studio використовується для:

- створення Robot'a;
- редагування Robot'a;
- створення Version[-s];
- роботи з компонентами;
- створення BPMN-процесів;
- тестування (через Orchestrator);
- ініціації переведення Robot/Version у PROD;
- перегляду доступної історії та станів відповідно до прав користувача.

2.2. Orchestrator

Orchestrator є центральним control layer системи. Він відповідає за:

- authorization;
- policy enforcement;
- TEST/PROD;
- approval;
- Test Period;
- queues;
- scheduling;
- execution control;
- пріоритети;
- availability;
- audit;
- передачу команд Assistant.

2.3. Assistant

Assistant - desktop application/agent, встановлений на машині, де фактично виконується/працює Robot. Assistant:

- отримує Execution від Orchestrator;
- виконує (start/stop/pause тощо) Robot'а;
- працює з GUI/API/OS залежно від компонентів Robot'а;
- передає технічний результат виконання назад в Orchestrator.

Assistant не повинен самотійно обходити policy та authorization Orchestrator'а.

3. Studio

3.1. Загальна модель

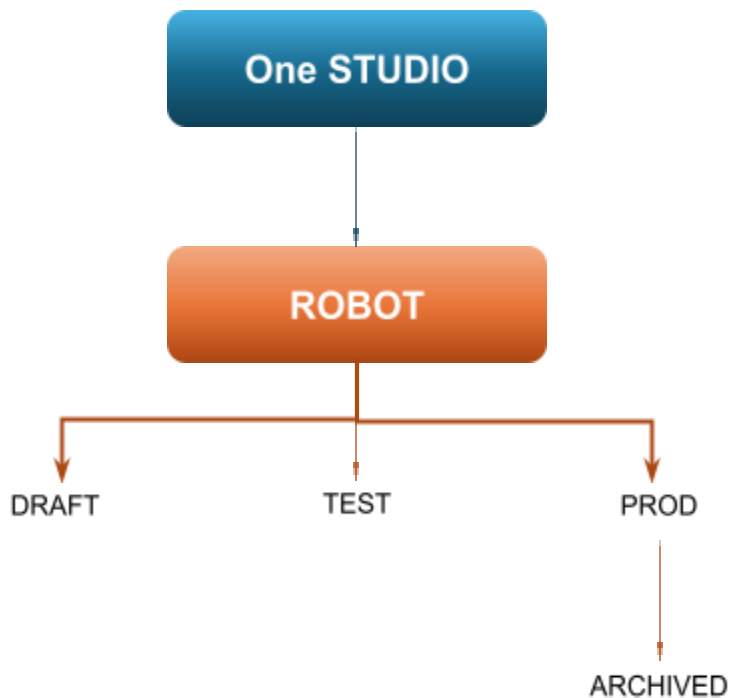
Існує **тільки одна Studio**.

Не створюються:

- TEST Studio;
- PROD Studio;
- окремі Studio для різних lifecycle.

TEST та PROD є логічними контекстами життєвого циклу та виконання, які контролюються Orchestrator'ом.

Studio працює з одним і тим самим Robot'ом протягом усього його життєвого циклу.



3.2. Рівні Studio

Level 1: BPMN

BPMN використовується для опису бізнес-процесу. В загальному виигляді BPMN складається з Task[-s]. Task - це **бізнес-задача**, яку виконує:

- людина;
- Robot;
- інша роль.

Task не є окремою низькорівневою дією Robot'а.

Level 2: Robot

Robot створюється з компонентів. Компоненти можуть працювати на рівнях:

- UI;
- API;
- Graph;
- OS;
- інше.

Level 3: Code

Embedding arbitrary code наразі не входить до поточного scope.

3.3. Custom components

Користувач повинен мати можливість створювати власні складені компоненти на базі базових компонентів. Наприклад, замість готового вузького:

Login to BAS

повинна бути можливість створити універсальну логіку:

Login to [selected system]

або складений користувацький компонент.

4. Robot як основна сутність

Robot - основна сутність, що представляє конкретну автоматизацію.

Robot має стабільну identity та власну історію Versions.

4.1. Основні атрибути Robot'a

Атрибут	Опис
Robot ID	Незмінний унікальний ідентифікатор
Name	Назва Robot'a
Description	Опис призначення Robot'a
Owner	Відповідальна особа/роль
Created By	Користувач, який створив Robot'a
Created At	Дата та час створення
Updated By	Користувач, який востаннє змінив Robot'a
Updated At	Дата та час останньої зміни
Lifecycle	DRAFT / TEST / PROD / ARCHIVED
Authorization	Поточний стан права на виконання
Availability	ENABLED / DISABLED / SUSPENDED
Health	HEALTHY / WARNING / ERROR / OFFLINE
Active Production Version	Посилання на конкретну активну PROD Version
Version[-s]	Набір усіх Version цього Robot'a

4.1.1. Robot ID

Robot ID генерується один раз під час створення Robot'a та залишається незмінним протягом усього життєвого циклу Robot'a.

Robot ID:

- не змінюється при перейменуванні Robot'a;
- не змінюється при створенні нової Version;
- використовується для однозначної ідентифікації Robot'a;
- використовується в Audit та Execution.

Кожна Version має власний порядковий номер у межах Robot'a.

Для зручної ідентифікації Version у користувацькому інтерфейсі використовується складений ідентифікатор у форматі:

Robot ID.Version Number

Наприклад:

Robot ID = 184

Version 1 → 184.1

Version 2 → 184.2

Version 3 → 184.3

Частина 184 визначає ID Robot'a, а частина після крапки визначає конкретну Version цього Robot'a.

Зміна Version не створює нового Robot'a та не змінює Robot ID.

Що маєтись на увазі: фізично в сховищі буде **дві повні версії робота**, але це **не два різні Robot'и** з точки зору розробника. Тобто:

Robot ID = 184

184.1

└─ повний стан робота:

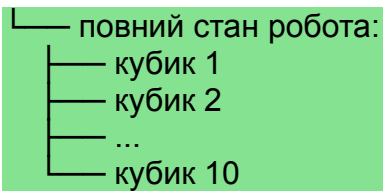
├─ кубик 1

├─ кубик 2

├─ ...

└─ кубик 10

184.2



Тому треба розрізняти два поняття

Логічно: це **один Robot 184**, у якого є версії 184.1, 184.2.

Фізично в сховищі: зберігаються **дві незалежні повні сутності/снапшоти**:

Robot 184
Version 1 → повна копія
Version 2 → повна копія

Саме це маєтись на увазі під «Зміна Version не створює нового Robot'а». Тобто, **не створюється новий Robot ID = 185**.

Але **створюється новий запис/об'єкт Version 2**, який містить повну копію **Robot 184** зі змінами.

Якщо зовсім просто: Розробник у Studio бачить:

Робот 184
└─ 184.1
└─ 184.2

А не:

Робот 184
Робот 185

При цьому 184.1 і 184.2 - **повністю самодостатні**. Якщо в 184.2 змінився тільки 4-й кубик, ми все одно зберігаємо всі 10 кубиків.

Це дасть простий rollback:

PROD зараз → 184.2

rollback

PROD → 184.1

Нічого не копіюємо назад і не відновлюємо окремі зміни - просто перемикаємо активну версію.

Отже: на диску/в сховищі фактично буде 2 повні роботи - але в системній моделі це 1 Robot із 2 Version.

4.1.2. Metadata Robot'a

Robot повинен зберігати інформацію про власний життєвий цикл та походження.

Мінімально:

Robot ID
Name
Description
Owner

Created By
Created At

Updated By
Updated At

Lifecycle
Authorization
Availability
Health

*Active Production Version**

Важливо

Необхідно розділяти:

- хто створив **Robot'a**;
- хто створив **Version**;
- хто виконав **release**;
- хто виконав **approval**;
- хто запросив **Execution**.

Це різні події та різні аудиторські дані.

* - **Active Production Version** визначає номер Version, яка на поточний момент використовується як Production Version для цього Robot'a. Наприклад:

Robot ID: 184

Active Production Version: 2

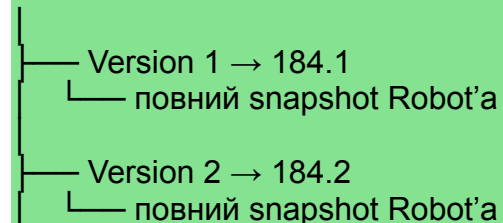
У користувацькому інтерфейсі (Розробника в Студії та Адміністратора в Оркестраторі) це може відображатися як: **Robot 184.2**

4.1.3. Version

Version - це повністю збережений та незмінний snapshot конкретного стану Robot'a. Один Robot може мати необмежену кількість Versions. Кожна Version містить **повну Definition / Configuration Robot'a**, а не тільки зміни відносно попередньої Version.

Наприклад:

Robot ID = 184





Version 2 не залежить від Version 1 для свого виконання. Вона містить повну реалізацію Robot'a на момент створення Version. Після створення Version її Definition / Configuration не змінюється.

Якщо необхідно змінити Robot'a таким чином, що зміна впливає на його виконання, створюється нова Version з новим Version Number та повною копією поточного стану Robot'a з урахуванням внесених змін.

Robot ID при створенні нової Version не змінюється.

4.1.4. Versioning rules

Нова Version створюється при збереженні змін, які можуть впливати на виконання Robot'a.

При створенні нової Version система:

1. бере повний стан поточної Version;
2. створює його повну копію;
3. застосовує внесені зміни;
4. присвоює новій Version наступний Version Number;
5. зберігає нову Version як незалежний незмінний snapshot.

Нова Version не повинна зберігати лише перелік змінених елементів або залежати від попередньої Version для відтворення власної Definition / Configuration.

До execution-affecting змін належать:

- логіка;
- code;
- components;
- sequence;
- conditions;
- parameters;
- variables;
- triggers;
- системи;
- ресурси;
- connection/configuration, якщо вони впливають на execution;
- інші execution-affecting settings.

Metadata-only changes

Окремо повинно бути визначено, які зміни вважаються metadata-only і не створюють нову Version. Це залишається design decision!!!

4.1.5. Структура Version[-s]

Мінімальна структура:

Поле	Опис
Robot ID	Robot, до якого належить Version
Version Number	v1, v2, ... vn
Created By	Хто створив Version
Created At	Дата та час створення
Change Description*	Опис змін
Release State	Стан release

Authorization	Authorization саме цієї Version
Definition / Configuration	Повний snapshot реалізації Robot'а
Definition Hash	Контроль цілісності повного snapshot'а
Released By	Хто виконав release
Released At	Дата та час release

* - приклад Change Description:

Robot 184.1
Change Description: —

Robot 184.2
Change Description: змінено блок 4 - додано перевірку наявності документа

Robot 184.3
Change Description: додано обробку помилки API

Release State

Пропоную використати наступні значення:

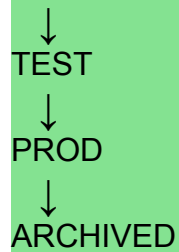
- DRAFT
- TESTING
- RELEASED
- SUPERSEDED
- REJECTED

Release State Version та Lifecycle Robot - **не одне й те саме. Це різні сутності.**

4.1.6. Lifecycle Robot'а

Lifecycle визначає етап життєвого циклу Robot'а.

DRAFT



DRAFT

Robot створений або налаштовується.

TEST

Robot знаходиться у тестуванні. Developer може:

- редагувати Robot'а;
- створювати Versions;
- запускати Robot;'а
- запускати окремі частини;
- повторювати executions;
- аналізувати результати.

PROD

Robot переведений у production lifecycle. **PROD не означає автоматично APPROVED.**

Можливий стан:

Lifecycle = PROD
Authorization = PENDING_APPROVAL

У такому випадку execution заборонений.

ARCHIVED

Robot більше не використовується. Він зберігається для:

- історії;
- audit;
- аналізу;
- відновлення інформації.

Execution ARCHIVED Robot заборонений.

4.1.7. Authorization

Authorization - окремий вимір стану Robot'a/Version.

Значення:

Status	Значення
NOT_REQUIRED	Погодження не потрібне
PENDING_APPROVAL	Очікується погодження
APPROVED	Погодження надано
REJECTED	Погодження відхилено
REVOKED	Раніше наданий дозвіл відкликано
EXPIRED	Термін дії дозволу завершився

Ключове правило

Production authorization прив'язується до **конкретної Version**. Наприклад:

*RBT-184.27**

Lifecycle: PROD

Authorization: APPROVED

* - Robot ID.Version Number

Якщо створена v28:

184.27 → APPROVED

184.28 → NOT APPROVED

Approval v27 не переноситься автоматично на v28.

4.1.8. Availability

Availability відповідає на питання: чи дозволено Robot'а виконувати зараз?

Значення:

- ENABLED
- DISABLED
- SUSPENDED

ENABLED

Robot може виконуватися, якщо всі інші перевірки пройдені.

DISABLED

Robot навмисно вимкнений.

SUSPENDED

Robot тимчасово призупинений через:

- інцидент;
- системне правило;
- security issue;
- іншу визначену policy.

Availability незалежна від Authorization.

Наприклад:

Lifecycle: PROD

Authorization: APPROVED

Availability: DISABLED

є валідним станом.

4.1.9. Health

Health описує технічний стан. Значення:

- HEALTHY
- WARNING
- ERROR
- OFFLINE

Health не повинен автоматично змінювати lifecycle.

Наприклад:

Lifecycle: PROD

Authorization: APPROVED

Availability: ENABLED

Health: ERROR

є валідною комбінацією.

4.1.10. Active Production Version

Для кожного Robot'а повинна бути однозначно визначена Active Production Version.

Active Production Version - це конкретна Version цього Robot'а, яка в поточний момент використовується для Production execution.

Active Production Version не є окремим статусом.

Наприклад:

Robot ID: 184

Version 1 → 184.1

Version 2 → 184.2

Version 3 → 184.3

Active Production Version = 2

У користувацькому інтерфейсі:

Active PROD = 184.2

Створення нової Version не повинно автоматично змінювати Active Production Version.

Наприклад:

184.2 → Active PROD

184.3 → TEST / development

При цьому обидві Version зберігаються повністю та незалежно одна від одної.

4.1.11. Rollback

Rollback - це повернення Active Production Version до раніше створеної Version Robot'a.

Оскільки кожна Version є повним незмінним snapshot'ом Robot'a, rollback не потребує відновлення окремих змінених блоків або застосування зворотних змін.

Наприклад:

Robot 184

184.1

184.2 ← попередня стабільна

184.3 ← Active PROD

Після rollback:

Robot 184

184.1

184.2 ← Active PROD

184.3

Rollback:

- не змінює жодну існуючу Version;
- не створює нову Version;
- не змінює Definition існуючих Version;
- змінює Active Production Version;

- повинен бути зафіксований в Audit.

Таким чином, rollback фактично є вибором раніше збереженого повного snapshot'a Robot'a для Production execution.

4.1.12. Test Period

Test Period - окремий дозвіл на виконання Robot'a у TEST протягом визначеного часу. Він потрібен **лише якщо TEST Approval Required = ON**.

Основні параметри

Параметр	Опис
ID	ID Test Period
Robot ID	Robot
Approved By	Admin
valid_from	Початок
valid_to	Завершення
Status	Статус періоду
Revoke	Можливість дострокового відкликання

Можливі тривалості:

- 1h;
- 3h;
- 1d;
- 2d;
- 1 week;
- custom.

Набір значень конфігурується Admin'ом.

4.1.13. TEST execution

Robot у TEST повинен дозволяти Developer'у виконувати багаторазові executions.

Наприклад:

TEST Period = 1 day

184.10

184.11

184.12

Execution #1

Execution #2

...

Execution #800

Новий approval перед кожним execution **не потрібен**. Під час активного Test Period Developer може:

- створювати нову Version на основі поточного стану Robot'а;
- тестувати окремі Version;
- тестувати окремі частини Version;
- виконувати багаторазові executions;
- аналізувати результати executions.

4.1.14. TEST Period expiry

Після завершення Test Period:

New TEST execution → BLOCKED

Для вже запущених Execution поведінка визначається Admin policy:

ALLOW_TO_FINISH: вже запущений Execution завершується сам, не примусово.

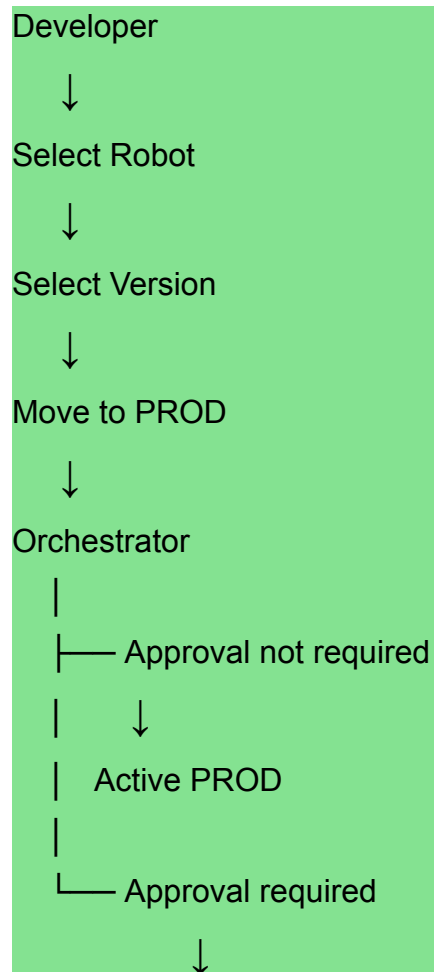
Новий - не починається.

TERMINATE: вже запущений Execution завершується примусово. Нові - не починаються.

Admin може відкликати активний Test Period достроково.

4.1.15. TEST → PROD

Developer ініціює переведення Robot'a у PROD.





Алгоритм дій

1. Developer обирає Robot'a
2. Developer обирає конкретну Version
3. Developer натискає Move to PROD
4. Studio передає до Orchestrator:
 - 4.1. Robot ID
 - 4.2. Version Number
 - 4.3. користувача
 - 4.4. час
 - 4.5. параметри request
5. Orchestrator перевіряє PROD policy
6. Якщо approval не потрібен - Version може стати Active Production Version
7. Якщо approval потрібен - Authorization = PENDING_APPROVAL
8. Execution заблокований до рішення
9. Admin виконує APPROVE або REJECT
10. APPROVE → Version може стати Active Production Version
11. REJECT → execution заблокований
12. Всі дії потрапляють в Audit

4.1.16. Execution

Execution - **окрема сутність**, яка представляє конкретний запуск конкретної Version. Це принципово важливо - Robot не повинен мати статус:

RUNNING

як характеристику самого Robot'а.

RUNNING - статус конкретного Execution'а.

Один Robot може мати:

Execution #1001 → Robot 184.26 → RUNNING

Execution #1002 → Robot 184.26 → COMPLETED

Execution #1003 → Robot 184.27 → QUEUED

Execution #1004 → Robot 184.25 → FAILED

одночасно.

4.1.17. Структура Execution

Поле	Опис
Execution ID	Унікальний ID запуску
Robot ID	Robot
Version Number	Конкретна Version
Lifecycle	TEST / PROD
Requested By	Хто запросив запуск
Requested At	Коли запросив
Assistant	Який Assistant (десктопний застосунок)
Host	На якій машині
Started At	Початок виконання

Finished At	Завершення
Execution Status	Статус запуску
Result	Результат
Error	Помилка, якщо є

Ключове правило: кожен Execution повинен однозначно посилатися на *Robot ID.Version Number*.

Наприклад:

Robot ID = 184

Version Number = 2

означає виконання повного snapshot **Robot'а 184.2**.

Це дозволяє визначити, **яка саме реалізація Robot'а фактично виконувалась**.

4.1.18. Execution Status

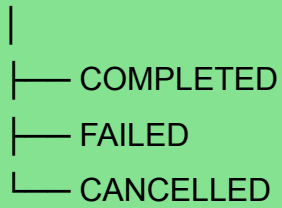
Status	Значення
QUEUED	Execution поставлено в чергу
RUNNING	Виконується
PAUSED	Призупинено
COMPLETED	Успішно завершено
FAILED	Завершено з помилкою
CANCELLED	Скасовано

Базовий flow:

QUEUED



RUNNING

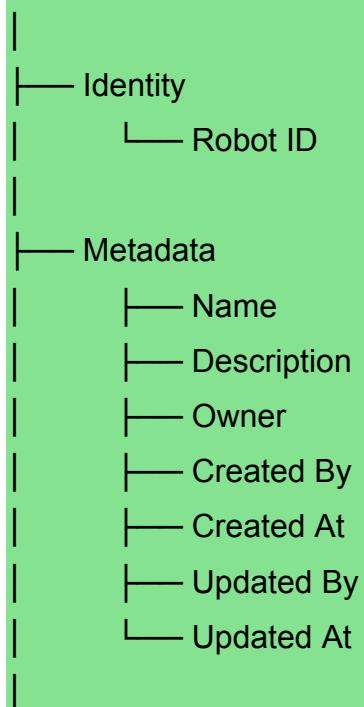


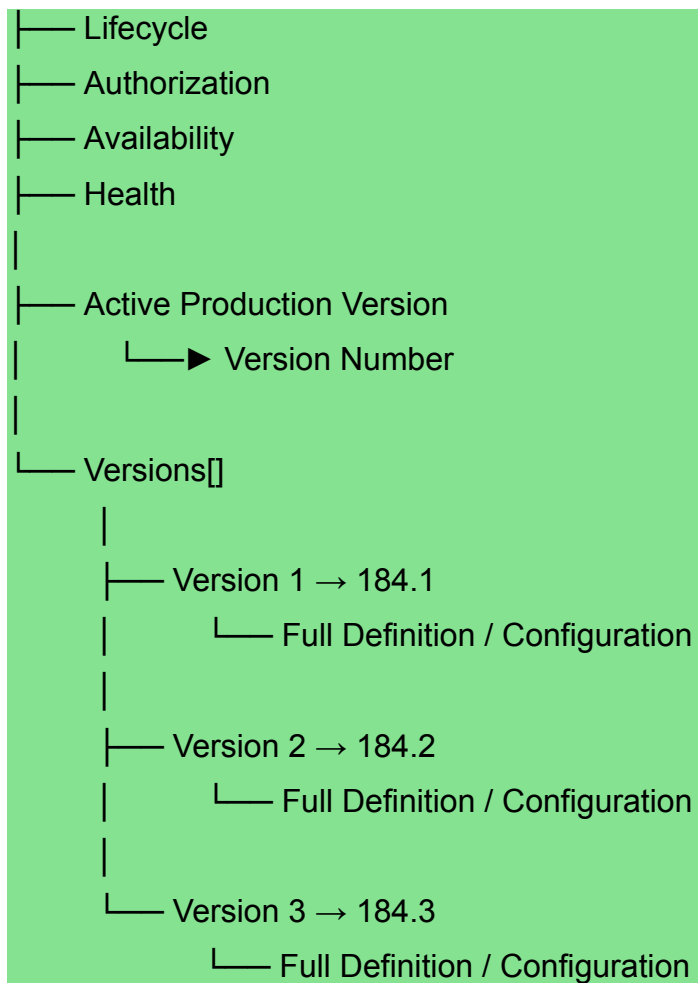
PAUSED може бути проміжним станом. Окремо потрібно визначити поведінку RUNNING Execution при:

- DISABLED;
- SUSPENDED;
- REVOKED;
- втраті Assistant;
- втраті connection.

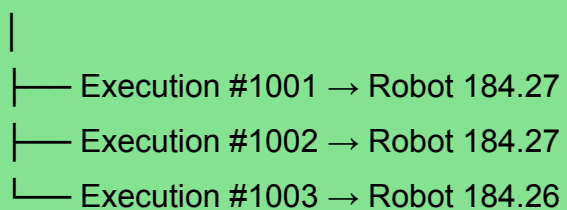
4.2. Повна модель Robot → Version → Execution

ROBOT





EXECUTION[]



Ключовий принцип: кожна Version є повним незалежним snapshot'ом Robot'a. Version не зберігає лише зміни відносно іншої Version та не потребує попередньої Version для відтворення власної Definition.

4.3. Принцип розділення станів

Сутність / атрибут	Питання
Robot	Що це за автоматизація?
Robot ID	Який саме Robot?
Version	Яка конкретна повністю збережена реалізація Robot'а?
Active Production Version	Яка Version зараз активна у PROD?
Lifecycle	На якому етапі життєвого циклу Robot?
Release State	На якому етапі release знаходиться Version?
Authorization	Чи дозволено виконання?
Availability	Чи дозволено виконання зараз?
Health	Який технічний стан?
Execution	Який конкретний запуск?
Execution Status	Що зараз відбувається з цим запуском?

Не допускається зводити всі ці поняття в один статус!!!

4.4. Visibility

Visibility та Permission - різні поняття.

Користувач може:

- бачити Robot'а;
- але не мати права його редагувати;
- бачити Version;
- але не мати права її запускати;
- бачити PROD Robot'а;
- але не мати права виконання.

Базова модель Visibility:

Object	Regular User	Developer	Admin
DRAFT Robot	Hi	Так	Так
TEST Robot	Hi	Так	Так
PROD Robot	Так, за RBAC	Так	Так
ARCHIVED Robot	Hi / policy	Так	Так
TEST Version	Hi	Так	Так
PROD Version	Так, за RBAC	Так	Так
TEST Execution	Hi / власні	Так	Так
PROD Execution	Так / дозволені	Так / дозволені	Так
Approval	Статус	Статус	Так
Test Period	Hi	Власний/requested	Так
Audit	Hi	Обмежено	Так

4.5. Permissions

Базовий перелік permissions:

- **Robot**
 - ROBOT_VIEW
 - ROBOT_CREATE
 - ROBOT_EDIT
 - ROBOT_DELETE
 - ROBOT_TEST
 - ROBOT_MOVE_TO_PROD
 - ROBOT_ARCHIVE
- **Version**
 - VERSION_VIEW
 - VERSION_CREATE
 - VERSION_EDIT

Фактична Version є незмінною після створення; VERSION_EDIT може використовуватися для draft/version metadata до моменту фіксації.

- **Execution**
 - EXECUTION_VIEW
 - EXECUTION_RUN
 - EXECUTION_CANCEL
- **Production**
 - PROD_APPROVE
 - PROD_REJECT
 - PROD_REVOKE
- **TEST**
 - TEST_PERIOD_REQUEST
 - TEST_PERIOD_APPROVE
 - TEST_PERIOD_REVOKE
- **Availability**
 - ROBOT_ENABLE
 - ROBOT_DISABLE
 - ROBOT_SUSPEND
 - ROBOT_RESUME
- **Administration**
 - POLICY_VIEW
 - POLICY_EDIT
 - AUDIT_VIEW

Фінальна RBAC-матриця є окремим design decision.

4.6. TEST → PROD interactions

TEST Robot не повинен напряму обходити Orchestrator для взаємодії з PROD Robot.

Базова policy:

TEST → PROD = DENY ALL

Можуть існувати exceptions.

Наприклад:

RBT-TEST-001 → RBT-PROD-015 = READ

RBT-TEST-002 → RBT-PROD-020 = CALL + READ

Доступні permissions:

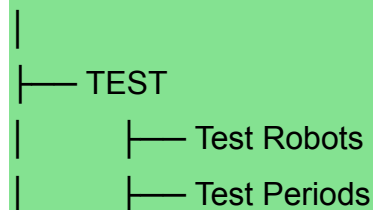
Permission	Значення
READ	Читання визначених даних/стану
CALL	Виклик визначеного PROD Robot'a
WRITE	Дозволений запис/зміна
TRIGGER	Запуск/тригер
ALL	Всі взаємодії дозволені
DENY ALL	Всі взаємодії заборонені

Точна семантика кожного permission повинна бути визначена окремо.

4.7. Orchestrator

Orchestrator - центральний компонент контролю:

ORCHESTRATOR





Фізично це буде одна інсталяція.

4.7.1. Функції Orchestrator

Orchestrator повинен:

- приймати Robot/Version information від Studio;
- зберігати authorization;
- перевіряти policy;
- контролювати Test Period;
- контролювати PROD approval;
- контролювати Availability;
- контролювати TEST → PROD interactions;
- керувати queue;
- керувати scheduling;
- визначати priority;
- передавати Execution Assistant;
- блокувати несанкціоновані executions;
- вести Audit.

Ключовий принцип:

Admin decides → Orchestrator controls/enforces → Assistant / Robot executes.

4.7.2. Pre-run checks

Перед кожним Execution Orchestrator повинен перевірити:

1. Robot існує
2. Version існує
3. Version належить цьому Robot'у
4. Lifecycle дозволяє execution
5. Authorization дозволяє execution
6. Authorization не EXPIRED / REVOKED
7. Availability = ENABLED
8. Robot не SUSPENDED
9. Для TEST існує чинний Test Period, якщо policy цього вимагає
10. Assistant доступний
11. Host доступний
12. Необхідні ресурси доступні
13. TEST → PROD policy не порушена
14. Користувач має EXECUTION_RUN
15. Scheduling policy дозволяє запуск
16. Concurrency limits дозволяють запуск
17. Інші system/security policies дозволяють запуск

Якщо будь-яка критична перевірка не пройдена:

Execution → BLOCKED

і причина повинна бути зафіксована в Audit.

4.7.3. Priority

PROD Executions мають вищий пріоритет за TEST Executions. Наприклад:

100 TEST jobs
+
1 PROD job

Orchestrator повинен забезпечити обробку PROD відповідно до встановленої scheduling policy. При цьому повинні бути визначені:

- concurrency;
- resource limits;
- starvation prevention;
- queue policy;
- resource locking;
- max parallel executions;
- поведінка при перевантаженні Assistant.

4.7.4. Admin Panel

Admin Panel знаходиться в Orchestrator. Admin повинен мати можливість налаштовувати:

TEST

- TEST Approval Required ON/OFF;
- дозволені Test Period;
- максимальну тривалість Test Period;
- поведінку після expiry:
 - ALLOW_TO_FINISH;

- TERMINATE.

PROD

- PROD Approval Required ON/OFF;
- approval;
- reject;
- revoke.

Availability

- Enable;
- Disable;
- Suspend;
- Resume.

TEST → PROD

- Global policy;
- exceptions для конкретних Robot.

Scheduling

- priority;
- concurrency;
- resource limits;
- queue policy.

Audit

- перегляд audit'y;
- фільтрація;
- пошук.

4.7.5. Audit

Orchestrator повинен вести Audit усіх критичних дій.

Robot

- створення;
- редагування;
- перейменування;
- зміна Owner'а;
- lifecycle transition;
- archive.

Version

- створення;
- release;
- promotion;
- production activation;
- supersede;
- rollback.

Authorization

- request;
- approve;
- reject;
- revoke;
- expire.

Test Period

- create;
- approve;

- revoke;
- expire.

Availability

- enable;
- disable;
- suspend;
- resume.

Execution

- request;
- block;
- queue;
- start;
- pause;
- complete;
- fail;
- cancel.

Policy

- створення;
- зміна;
- активація;
- деактивація.

Audit Event

Мінімально:

- Who
- When

- Action
- Robot ID
- Version Number
- Previous State
- New State
- Reason
- Execution ID
- Assistant
- Host
- Lifecycle

4.8. Assistant

Використовується **один Assistant** для TEST та PROD.

Assistant отримує Execution Context:

TEST

або

PROD

Assistant:

- отримує execution від Orchestrator'a;
- завантажує конкретну Version;
- виконує її;
- працює з цільовою машиною;
- повертає результат;
- передає технічні події виконання.

Assistant не повинен мати можливості обійти:

- authorization;
- availability;
- policy;
- approval;
- queue.

5. Приклад lifecycle

Розглянемо Robot'a:

Robot ID: *RBT-184*

Name: *Обробка рахунків*

Крок 1

Developer створює Robot.

Lifecycle = DRAFT

Зберігається:

Created By

Created At

Крок 2

Developer переводить Robot'a у TEST.

Lifecycle = TEST

Крок 3

Створюється:

184.10

Крок 4

Developer тестує Robot'a.

Створюються:

Execution #1001

Execution #1002

...

Execution #1100

Крок 5

Створюються:

184.11

184.12

Крок 6

Developer тестує v12.

184.12

Authorization = TEST authorization

Крок 7

Developer вибирає:

RBT-184.12

і натискає:

Move to PROD

Крок 8

Orchestrator перевіряє PROD policy.

Якщо approval потрібен:

Lifecycle = PROD

Authorization = PENDING_APPROVAL

Крок 9

Admin погоджує:

Authorization = APPROVED

Крок 10

Active Production Version = v12

Крок 11

Regular User може бачити PROD Robot'a та запускати його відповідно до RBAC.

Крок 12

Developer починає роботу над наступною версією:

v13

При цьому:

Active PROD = v12

i approval v12 **не переноситься автоматично на v13.**

6. Приклад TEST

Robot:

RBT-184.1

Lifecycle = TEST

Policy:

TEST Approval Required = ON

Admin надає:

Test Period

valid_from = 10:00

valid_to = 18:00

Developer:

- змінює Robot'a;
- створює v10;
- створює v11;
- створює v12;
- запускає Robot'a 800 разів;
- тестує окремі частини;
- аналізує failures.

Новий approval перед кожним execution не потрібен.

О 18:00:

Test Period = EXPIRED

Нові executions:

BLOCKED

7. Data model

Robot

```
Robot {  
  RobotID  
  Name  
  Description  
  Owner  
  
  CreatedBy  
  CreatedAt  
  UpdatedBy  
  UpdatedAt  
  
  Lifecycle  
  
  Authorization  
  Availability  
  Health  
  
  ActiveProductionVersion  
  
  Versions[]  
}
```

Version

```
Version {  
  RobotID  
  Version Number  
  
  CreatedBy  
  CreatedAt
```

```
ChangeDescription
ReleaseState
Authorization

Definition
DefinitionHash

ReleasedBy
ReleasedAt
}
```

TestPeriod

```
TestPeriod {
  ID
  RobotID

  ApprovedBy

  ValidFrom
  ValidTo

  Status
  RevokedAt
}
```

Execution

```
Execution {
  ExecutionID

  RobotID
  VersionID
}
```

Lifecycle

RequestedBy

RequestedAt

Assistant

Host

Status

StartedAt

FinishedAt

Result

Error

}

8. Responsibility matrix

Дія	Developer	Orchestrator	Admin	Assistant
Створити Robot'a	Так	Зберігає / контролює	-	-
Редагувати Robot'a	Так	Контроль policy	-	-
Створити Version	Так	Зберігає	-	-
Тестувати	Ініціює	Дозволяє / блокує	Test Period, якщо потрібно	Виконує
Move to PROD	Ініціює	Обробляє	-	-
Approve PROD	-	Enforce	Так	-
Reject PROD	-	Enforce	Так	-
Production execution	Запитує за RBAC	Авторизує / Queue	Policy	Виконує
Disable Robot	Permission	Enforce	Permission	-
Suspend Robot	Permission	Enforce	Permission	-
Audit	-	Так	Перегляд	Передає runtime data

9. Acceptance Criteria

Система вважається такою, що відповідає ТЗ, якщо:

1. Robot має незмінний Robot ID
2. Robot має Name, Description, Owner
3. Robot має Created By / Created At
4. Robot має Updated By / Updated At
5. Version є окремою сутністю
6. Robot має набір Versions
7. Version має Version Number у межах конкретного Robot'a
8. Кожна Version містить повний snapshot Definition / Configuration Robot'a
9. Version не залежить від попередньої Version для відтворення або виконання
10. Version має Created By / Created At
11. Version є незмінною після створення
12. Нова execution-affecting зміна створює нову Version, яка є повним snapshot'ом Robot'a з урахуванням цієї зміни
13. Active Production Version однозначно визначає конкретну Version за Robot ID.Version Number
14. Active Production Version не є окремим lifecycle status
15. Lifecycle підтримує DRAFT / TEST / PROD / ARCHIVED
16. Authorization є окремим виміром
17. Availability є окремим виміром
18. Health є окремим виміром
19. PROD може мати PENDING_APPROVAL
20. PROD execution неможливий без необхідного approval
21. Approval прив'язаний до конкретної Version
22. Нова Version не успадковує автоматично PROD approval попередньої
23. TEST підтримує багаторазові executions
24. TEST не вимагає approval перед кожним execution
25. Test Period має valid_from / valid_to

26. Test Period може бути revoked
27. Після expiry нові TEST executions блокуються
28. Поведінка вже запущених executions після expiry задається policy
29. Один Assistant використовується для TEST та PROD
30. Orchestrator контролює execution
31. Assistant не обходить Orchestrator
32. TEST та PROD логічно розділені в Orchestrator
33. PROD має вищий priority за TEST
34. Execution є окремою сутністю
35. Execution посилається на конкретні Robot ID.Version Number
36. Regular User не бачить DRAFT/TEST за базовою моделлю
37. Developer бачить Robot'а відповідно до RBAC
38. Admin має повну visibility
39. TEST → PROD interaction контролюється policy
40. Критичні дії записуються в Audit
41. Rollback не створює нову Version
42. Rollback змінює Active Production Version
43. Rollback не змінює Definition жодної існуючої Version

10. Невирішені design decisions

Наступні питання поки не повинні бути «вигадані» в ТЗ як уже затверджені:

1. Які саме зміни є metadata-only
2. Чи можуть одночасно існувати кілька APPROVED PROD Versions
3. Чи може одночасно виконуватися кілька Versions одного Robot'a
4. Точний approval flow для rollback
5. Хто має право ініціювати rollback
6. Чи потребує rollback окремого approval
7. Чи можна відновлювати ARCHIVED Robot'a
8. Точна семантика READ / CALL / WRITE / TRIGGER
9. Чи може TEST викликати PROD виключно через Orchestrator API
10. Умови автоматичного SUSPENDED
11. Поведінка RUNNING Execution після DISABLED
12. Поведінка RUNNING Execution після SUSPENDED
13. Поведінка RUNNING Execution після REVOKED
14. Фінальна RBAC-матриця
15. Scheduling algorithm
16. Concurrency model
17. Starvation prevention
18. Resource locking
19. Resource reservation
20. Deployment нової PROD Version без downtime
21. Точний UX lifecycle/version statuses у Studio
22. Поведінка working Version при одночасному Active Production Version
23. Точна модель Version Release State
24. Чи потрібен окремий DEPRECATED / RETIRED lifecycle
25. Чи має TEST Version окремий authorization object, чи тільки Test Period
26. Чи може Test Period охоплювати нові Versions, створені після його approval
27. Що відбувається при створенні нової Version під час активного PROD

28. Яким чином Orchestrator визначає конкретний runtime package/Definition для Assistant

11. Фінальна концептуальна модель

У системі необхідно чітко розділяти:

IDENTITY



Robot ID



VERSIONS

184.1 / 184.2 / 184.3 / ...



FULL IMMUTABLE SNAPSHOT



LIFECYCLE / AUTHORIZATION / AVAILABILITY / HEALTH

Для кожного Robot'a:

Robot

RobotID = 184



├── Version 1



Robot ID = 184



Number = 1



Display = 184.1



Definition = FULL SNAPSHOT



├── Version 2



Robot ID = 184



Number = 2



Display = 184.2



Definition = FULL SNAPSHOT



└─ Version 3
 Robot ID = 184
 Number = 3
 Display = 184.3
 Definition = FULL SNAPSHOT

Одна Version може бути визначена як Active Production Version.

Execution завжди посилається на конкретну пару: **Robot ID.Version Number**

Наприклад: **Robot 184.2**

Основний принцип:

Robot визначає, що це за автоматизація.

Version визначає конкретний повністю збережений стан реалізації цієї автоматизації.

Version є незмінною та самодостатньою для виконання.

Lifecycle визначає етап життєвого циклу Robot'а.

Authorization визначає, чи дозволене виконання.

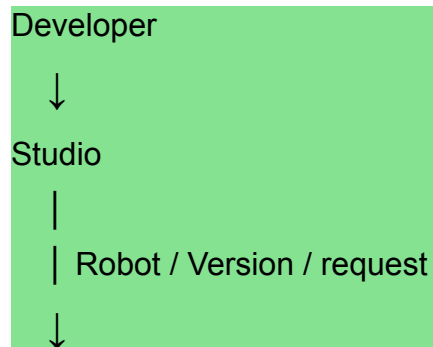
Availability визначає, чи дозволене виконання зараз.

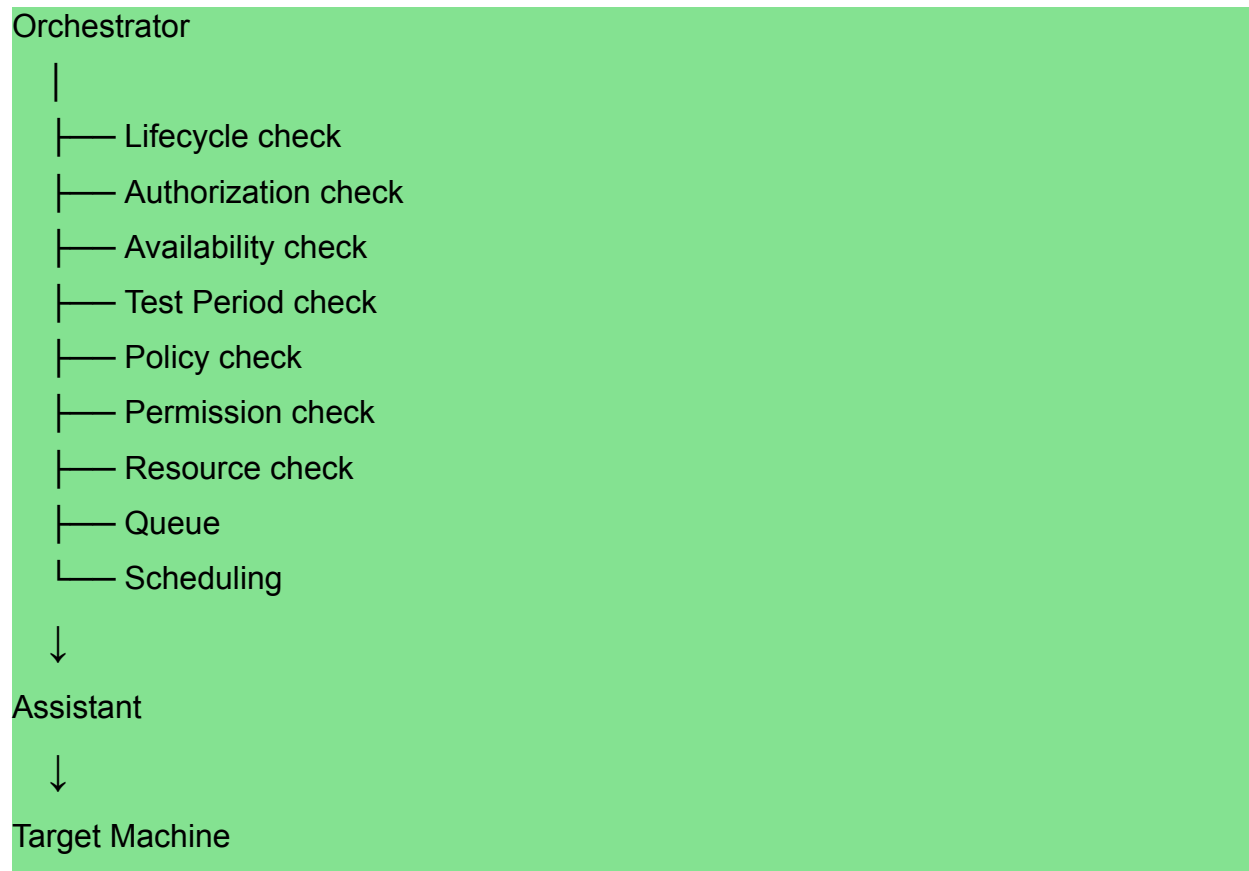
Health визначає технічний стан.

Active Production Version визначає, яка Version зараз працює як PROD.

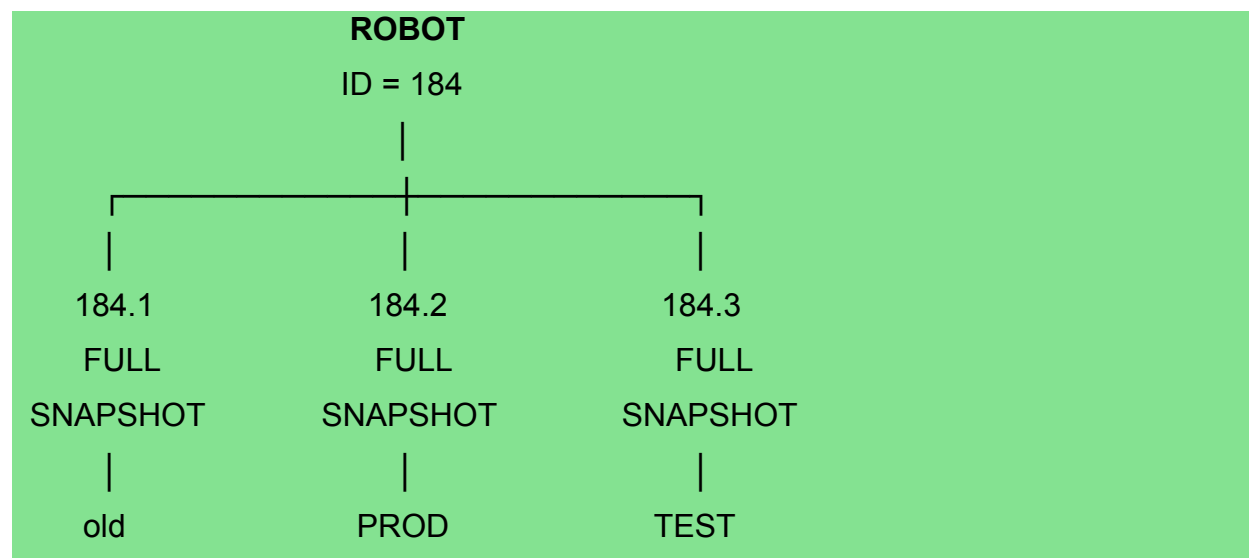
Execution визначає конкретний запуск конкретної Version.

І вся операційна модель зводиться до:





Отже, модель Robot'a:



Ми окремо зберігаємо Robot ID, окремо - Version Number, але розробнику/користувачу показуємо (Display) їх комбінацію “RobotID” + “.” + “Version Number”:

```
Robot
└── Robot ID = 184

Version
├── Robot ID = 184
├── Version Number = 27
└── Display = 184.27
```

Таким чином 184.2 - **не якийсь окремий технічний Version ID**, а зрозуміле представлення:

```
Robot ID = 184
Version = 2
```

При цьому в базі можна мати:

```
Robot
-----
ID = 184

RobotVersion
-----
RobotID = 184
Number = 1
Definition = [повний snapshot]

RobotVersion
-----
```

RobotID = 184

Number = 2

Definition = [повний snapshot]

RobotVersion

RobotID = 184

Number = 3

Definition = [повний snapshot]

Отже, ми беремо ідею про «копію повністю» та простий rollback, але **не робимо штучно новий Robot ID для кожної версії.**